

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования

«Санкт-Петербургский политехнический университет Петра Великого»
(ФГАОУ ВО СПбПУ)

Институт компьютерных наук и кибербезопасности
Высшая школа компьютерных технологий и информационных систем

ПРАКТИЧЕСКАЯ РАБОТА

По дисциплине «Объектно-ориентированное программирование»
8-Puzzle Game

Студент гр. 5140901/51501

Н.С.Пиюренко

подпись, дата

инициалы и фамилия

Оценка выполненной студентом работы:

Руководитель,
д.т.н, профессор ВШКТиИС.

Г.Ф.Малыхина

подпись, дата

инициалы и фамилия

Санкт-Петербург – 2025

САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО

Институт компьютерных наук и кибербезопасности

Высшая школа компьютерных технологий и информационных систем

ЗАДАНИЕ

на выполнение практической работы по дисциплине

«Объектно-ориентированное программирование»

студенту Пиюренко Никите Сергеевичу

группа: 5140901/51501

1. Тема работы: Создание программы на языке программирования Java, 8-Puzzle Game.
2. Срок сдачи студентом законченной работы: 05.12.2025

СОДЕРЖАНИЕ

1	Общие сведения	4
1.1	Идея программы.....	4
1.2	Основные правила.....	4
2	Реализация классов	6
2.1	Класс PuzzleState	6
2.2	Методы оценки состояния	6
2.3	Генерация состояний и проверка разрешимости.....	7
3	Алгоритм поиска решения	9
3.1	Реализация алгоритма A*	9
3.2	Восстановление пути решения.....	10
3.3	Оптимизация и корректность.....	11
4	Реализация графического интерфейса.....	12
4.1	Класс PuzzlePanel	12
4.2	Главное окно приложения.....	12
4.3	Элементы управления.....	13
5	Особенности реализации.....	14
5.1	Асинхронное выполнение поиска	14
5.2	Управление состоянием приложения	14
5.3	Управление процессом симуляции	15
6	Критерий завершения работы.....	16
	Приложение	17

1 Общие сведения

1.1 Идея программы

Данный программный продукт представляет собой интерактивную реализацию классической головоломки "Пятнашки" (8-Puzzle), разработанную с использованием языка программирования Java и библиотеки Swing для создания графического интерфейса пользователя. Основная концепция игры заключается в перемещении пронумерованных плиток на игровом поле размером 3x3 с целью достижения целевого расположения, где все числа упорядочены по возрастанию, а последняя клетка остается пустой.

Программа реализует алгоритм поиска A* (A-star) для нахождения оптимального решения головоломки из любого допустимого начального состояния. Алгоритм использует эвристическую функцию Манхэттенского расстояния для оценки стоимости достижения целевого состояния, что обеспечивает эффективный поиск кратчайшего пути решения. Эвристика рассчитывается как сумма расстояний каждой плитки от ее целевой позиции, что является допустимой оценкой, гарантирующей оптимальность решения.

Игровой процесс построен на принципах пошагового выполнения найденного решения, что позволяет пользователю наблюдать каждый шаг алгоритма и соответствующее перемещение плиток. Визуальное представление игрового поля обеспечивает интуитивно понятное восприятие состояния головоломки и процесса ее решения.

1.2 Основные правила

Для обеспечения корректной работы игровой механики были реализованы следующие фундаментальные правила и принципы. Игровое пространство представляет собой квадратное поле размером 3x3 клетки, содержащее восемь пронумерованных плиток и одну пустую клетку. Целевое состояние определено

как расположение, в котором числа от 1 до 8 упорядочены слева направо и сверху вниз, а пустая клетка находится в правом нижнем углу.

Механика перемещений позволяет сдвигать плитки только в направлении пустой клетки, что соответствует классическим правилам головоломки. Допустимыми действиями являются перемещения вверх, вниз, влево и вправо, но только если такое перемещение приводит к изменению положения пустой клетки. Каждое действие имеет единичную стоимость, что учитывается алгоритмом поиска при расчете общей длины пути решения.

Важным аспектом реализации является проверка разрешимости начальной конфигурации. Программа использует математический критерий, основанный на подсчете инверсий в последовательности чисел. Конфигурация считается разрешимой, если количество инверсий является четным числом, что соответствует теоретическим основам головоломки.

2 Реализация классов

2.1 Класс PuzzleState

Класс `PuzzleState` является фундаментальным компонентом программы, представляющим состояние игрового поля и инкапсулирующим всю логику работы с отдельными конфигурациями пазла. Класс реализует методы для определения возможных действий, применения перемещений и вычисления эвристических оценок.

Основные поля класса включают двумерный массив `board` для хранения текущего расположения плиток, координаты `blankRow` и `blankCol` пустой клетки, строку `action` описывающую последнее выполненное действие, ссылку `parent` на предыдущее состояние и целочисленное значение `cost` представляющее стоимость достижения текущего состояния. Константы `SIZE` и `GOAL` определяют размер поля и целевое расположение соответственно.

Система создания новых состояний реализована через конструкторы, обеспечивающие корректное наследование параметров и обновление координат пустой клетки. Метод `copyBoard` гарантирует создание независимых копий игрового поля, что предотвращает нежелательные побочные эффекты при модификациях состояний.

2.2 Методы оценки состояния

Ключевым компонентом алгоритма A^* является эвристическая функция, реализованная в методе `getManhattanDistance`. Данный метод вычисляет сумму Манхэттенских расстояний каждой плитки от ее целевой позиции. Для каждой непустой клетки определяется ее целевая позиция в решенном состоянии, после чего рассчитывается расстояние как сумма абсолютных разностей по строкам и столбцам.

Метод `getTotalCost` объединяет фактическую стоимость достижения состояния (количество выполненных перемещений) и эвристическую оценку

оставшегося пути. Эта сумма используется алгоритмом A^* для приоритизации состояний при поиске решения. Компаратор в PriorityQueue обеспечивает обработку состояний в порядке возрастания общей стоимости.

Система проверки достижения целевого состояния реализована в методе isGoal, который использует глубокое сравнение массивов для проверки соответствия текущего расположения целевому. Это гарантирует корректное определение завершающего состояния алгоритма поиска.

```
public int getManhattanDistance() { 1 usage
    int distance = 0;
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            int value = board[i][j];
            if (value != 0) {
                int targetRow = (value - 1) / SIZE;
                int targetCol = (value - 1) % SIZE;
                distance += Math.abs(i - targetRow) + Math.abs(j - targetCol);
            }
        }
    }
    return distance;
}
```

Рисунок 2.2.1 — Вычисление Манхэттенского расстояния

2.3 Генерация состояний и проверка разрешимости

Метод getPossibleActions анализирует текущее положение пустой клетки и определяет все допустимые направления перемещения. Проверка границ игрового поля обеспечивает корректность формируемого списка действий. Каждое действие представлено в виде строки, что упрощает последующую визуализацию и логирование.

Статический метод isSolvable реализует математический алгоритм проверки разрешимости конфигурации. Алгоритм основан на подсчете инверсий в линейной последовательности чисел, полученной обходом игрового поля слева направо и сверху вниз. Конфигурация считается разрешимой если количество

инверсий является четным числом, что соответствует математической теории ГОЛОВОЛОМКИ.

```
public static boolean isSolvable(int[][] board) { 1 usage
    int inversions = 0;
    List<Integer> flattened = new ArrayList<>();

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i][j] != 0) {
                flattened.add(board[i][j]);
            }
        }
    }

    for (int i = 0; i < flattened.size(); i++) {
        for (int j = i + 1; j < flattened.size(); j++) {
            if (flattened.get(i) > flattened.get(j)) {
                inversions++;
            }
        }
    }

    return inversions % 2 == 0;
}
```

Рисунок 2.3.1 — Проверка разрешимости

Метод generateRandomState создает случайные начальные конфигурации, гарантированно разрешимые. Алгоритм генерирует случайные перестановки чисел от 0 до 8 и проверяет их на разрешимость. В случае неразрешимой конфигурации выполняется рекурсивный вызов до тех пор, пока не будет найдена допустимая расстановка.

```
public static int[][] generateRandomState() { 2 usages
    List<Integer> numbers = new ArrayList<>(Arrays.asList(0, 1, 2, 3, 4, 5, 6, 7, 8));
    Collections.shuffle(numbers);

    int[][] board = new int[3][3];
    int index = 0;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            board[i][j] = numbers.get(index++);
        }
    }

    if (!isSolvable(board)) {
        return generateRandomState();
    }

    return board;
}
```

Рисунок 2.3.2 — Создание разрешимых начальных конфигураций

3 Алгоритм поиска решения

3.1 Реализация алгоритма A*

Класс `PuzzleSolver` содержит статический метод `solveAStar`, реализующий алгоритм поиска A* для нахождения оптимального пути от начального состояния к целевому. Алгоритм использует приоритетную очередь для обработки состояний в порядке возрастания их общей стоимости, что обеспечивает эффективный поиск кратчайшего пути.

```
public static List<PuzzleState> solveAStar(PuzzleState start) { 1 usage
    java.util.PriorityQueue<PuzzleState> openSet = new java.util.PriorityQueue<>(  
        java.util.Comparator.comparingInt(PuzzleState::getTotalCost)  
    );  
    java.util.Map<PuzzleState, Integer> gScore = new java.util.HashMap<>();  
    java.util.Map<PuzzleState, PuzzleState> cameFrom = new java.util.HashMap<>();  
  
    openSet.offer(start);  
    gScore.put(start, 0);  
  
    while (!openSet.isEmpty()) {  
        PuzzleState current = openSet.poll();  
  
        if (current.isGoal()) {  
            return reconstructPath(current);  
        }  
  
        for (String action : current.getPossibleActions()) {  
            PuzzleState neighbor = current.applyAction(action);  
            int tentativeGScore = gScore.get(current) + 1;  
  
            if (tentativeGScore < gScore.getOrDefault(neighbor, Integer.MAX_VALUE)) {  
                cameFrom.put(neighbor, current);  
                gScore.put(neighbor, tentativeGScore);  
  
                if (!openSet.contains(neighbor)) {  
                    openSet.offer(neighbor);  
                }  
            }  
        }  
    }  
  
    return Collections.emptyList();  
}
```

Рисунок 3.1.1 — Алгоритм поиска A*

Алгоритм поддерживает две основные структуры данных: `openSet` для хранения состояний, подлежащих обработке, и `gScore` для отслеживания лучшей известной стоимости достижения каждого состояния. Дополнительно используется карта `cameFrom` для восстановления пути решения после достижения цели.

На каждой итерации алгоритм извлекает состояние с наименьшей общей стоимостью из `openSet`. Если это состояние является целевым, выполняется восстановление пути. В противном случае генерируются все возможные соседние состояния и обновляются соответствующие записи в структурах данных.

3.2 Восстановление пути решения

Метод `reconstructPath` выполняет обратный проход от целевого состояния к начальному, используя информацию о родительских состояниях, сохраненную в карте `cameFrom`. Результирующий путь формируется как список состояний в порядке от начального к целевому, что обеспечивает удобство последующего пошагового воспроизведения.

```
private static List<PuzzleState> reconstructPath(PuzzleState goal) {  
    List<PuzzleState> path = new ArrayList<>();  
    PuzzleState current = goal;  
  
    while (current != null) {  
        path.add(index: 0, current);  
        current = current.getParent();  
    }  
  
    return path;  
}
```

Рисунок 3.2.1 — Обратный проход

Каждое состояние в пути решения содержит информацию о выполненном действии, что позволяет отображать пользователю последовательность перемещений, необходимых для решения головоломки. Это обеспечивает образовательную ценность программы, демонстрируя логику решения задачи.

3.3 Оптимизация и корректность

Использование Манхэттенского расстояния в качестве эвристической функции гарантирует допустимость и монотонность, что обеспечивает оптимальность найденного решения. Эвристика никогда не переоценивает действительную стоимость достижения цели, что является необходимым условием корректности алгоритма A^* .

Приоритетная очередь обеспечивает эффективное извлечение состояния с минимальной стоимостью, а хэш-карты предоставляют быстрый доступ к сохраненной информации о состояниях. Это позволяет алгоритму эффективно обрабатывать большое количество состояний без чрезмерных вычислительных затрат.

4 Реализация графического интерфейса

4.1 Класс PuzzlePanel

Класс `PuzzlePanel` наследуется от `JPanel` и отвечает за визуализацию текущего состояния игрового поля. Панель использует кастомную отрисовку для создания привлекательного интерфейса с закругленными плитками и градиентными цветами.

Размеры визуальных элементов жестко заданы константами `tileSize` и `margin`, что обеспечивает единообразие отображения. Метод `setBoard` позволяет обновлять отображаемое состояние и автоматически запускает перерисовку компонента.

В методе `paintComponent` реализована логика отрисовки каждой клетки игрового поля. Пустая клетка визуализируется как светло-серый квадрат, в то время как пронумерованные плитки отображаются с синим фоном и белыми цифрами. Использование `Graphics2D` и антиалиасинга обеспечивает высокое качество визуального представления.

4.2 Главное окно приложения

Класс `EightPuzzleSwing` является основным контейнером приложения и наследуется от `JFrame`. Он организует все компоненты интерфейса в единую структуру с использованием менеджера компоновки `BorderLayout`.

Центральную область окна занимает панель пазла, в то время как нижняя часть содержит панель управления, метку информации и область для отображения решения. Такая организация обеспечивает интуитивно понятное взаимодействие с программой и эффективное использование пространства окна.

Инициализация интерфейса выполняется в методе `initializeUI`, который создает все компоненты, устанавливает их свойства и добавляет в соответствующие контейнеры. Установка `setResizable(false)` гарантирует сохранение пропорций игрового поля при работе программы.

4.3 Элементы управления

Панель управления содержит четыре основные кнопки: "Решить" для запуска алгоритма поиска решения, "Перемешать" для генерации новой случайной конфигурации, "Сброс" для возврата к исходному состоянию и "Следующий шаг" для пошагового выполнения найденного решения.

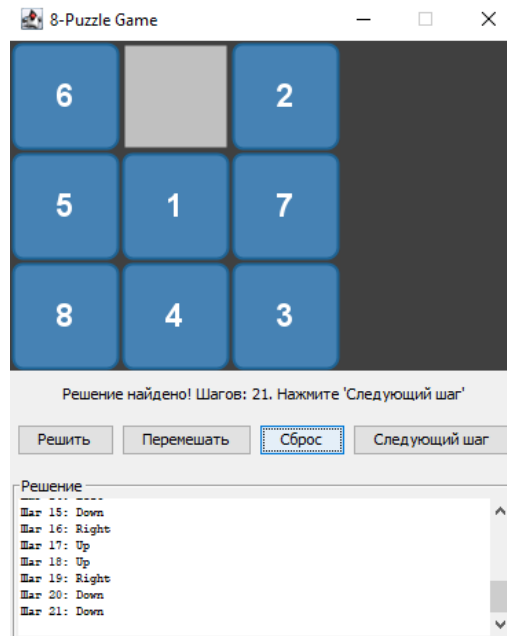


Рисунок 4.3.1 — Визуальное представление программы

Кнопка "Следующий шаг" активируется только после успешного нахождения решения и позволяет пользователю наблюдать последовательность перемещений плиток. Это обеспечивает интерактивное изучение процесса решения головоломки.

Метка `infoLabel` отображает текущий статус программы, включая сообщения о ходе поиска решения, количестве шагов и инструкции для пользователя. Текстовая область `solutionArea` показывает подробное описание пути решения в виде последовательности действий.

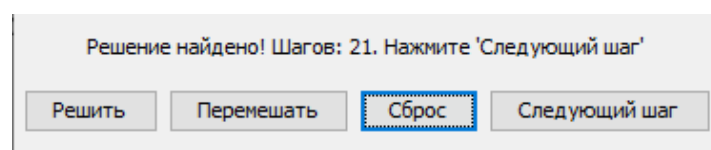


Рисунок 4.3.2 — Кнопки управления

5 Особенности реализации

5.1 Асинхронное выполнение поиска

Для предотвращения блокировки пользовательского интерфейса во время выполнения ресурсоемкого алгоритма поиска, решение головоломки выполняется в отдельном потоке с использованием класса `SwingWorker`. Это обеспечивает отзывчивость интерфейса и возможность отображения прогресса операции.

Метод `solvePuzzle` создает экземпляр `SwingWorker`, который выполняет поиск решения в фоновом потоке. По завершении поиска в потоке обработки событий обновляется интерфейс и активируются соответствующие элементы управления.

Такой подход позволяет пользователю продолжать взаимодействие с программой (например, просматривать текущее состояние) даже во время выполнения длительных вычислений. Это особенно важно для сложных конфигураций, требующих значительного времени поиска.

5.2 Управление состоянием приложения

Программа поддерживает несколько состояний: начальное, процесс поиска решения, наличие готового решения и процесс пошагового выполнения. Переходы между состояниями управляются через комбинацию флагов и переменных, отслеживающих текущий шаг решения.

Метод `shufflePuzzle` инициализирует новую случайную конфигурацию и сбрасывает все связанные с решением переменные. Метод `resetButton` обеспечивает возврат к последней начальной конфигурации без генерации новой случайной расстановки.

Система пошагового выполнения в методе `showNextStep` обновляет отображаемое состояние и соответствующую информацию в интерфейсе.

Автоматическое отключение кнопки "Следующий шаг" после завершения решения предотвращает возможные ошибки взаимодействия.

5.3 Управление процессом симуляции

Реализована система управления симуляцией с использованием Timer для пошагового выполнения действий агента с видимой задержкой. Это позволяет пользователю наблюдать процесс принятия решений и перемещения агента в реальном времени.

Симуляция автоматически отслеживает ограничение по количеству шагов (25 шагов) и прекращается при достижении этого лимита, что предотвращает бесконечное выполнение в случае заикливания политики. Это также обеспечивает сбор сравнительной статистики для различных политик.

6 Критерий завершения работы

Программа завершает процесс поиска решения при наступлении одного из следующих событий: достижение целевого состояния (успешное завершение), исчерпание всех возможных состояний без нахождения решения (неразрешимая конфигурация), или прерывание пользователем через закрытие приложения.

Пошаговое выполнение решения завершается когда достигнуто конечное состояние пути решения. После этого пользователь может начать новую игру путем перемешивания пазла или анализа найденного решения через текстовое описание.

Программа обеспечивает корректное освобождение ресурсов при завершении работы и устойчива к различным сценариям взаимодействия пользователя с интерфейсом.

Приложение

Программный код на языке Java, моделирующий игру «Пятнашки» или 8-Puzzle Game:

```
import javax.swing.*;
import java.awt.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

// Класс состояния пазла (остается практически без изменений)
class PuzzleState {
    private final int[][] board;
    private final int blankRow;
    private final int blankCol;
    private final String action;
    private final PuzzleState parent;
    private final int cost;

    public static final int SIZE = 3;
    public static final int[][] GOAL = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 0}
    };

    public PuzzleState(int[][] board) {
        this.board = copyBoard(board);
        this.action = "Initial";
        this.parent = null;
        this.cost = 0;
        int[] blankPos = findBlank();
        this.blankRow = blankPos[0];
        this.blankCol = blankPos[1];
    }

    private PuzzleState(int[][] board, String action, PuzzleState parent, int
newBlankRow, int newBlankCol) {
        this.board = board;
        this.action = action;
        this.parent = parent;
        this.cost = parent.cost + 1;
        this.blankRow = newBlankRow;
        this.blankCol = newBlankCol;
    }

    private int[] findBlank() {
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                if (board[i][j] == 0) {
                    return new int[]{i, j};
                }
            }
        }
        throw new IllegalArgumentException("Board must contain a blank cell (0)");
    }
}
```

```

private int[][] copyBoard(int[][] original) {
    int[][] copy = new int[SIZE][SIZE];
    for (int i = 0; i < SIZE; i++) {
        System.arraycopy(original[i], 0, copy[i], 0, SIZE);
    }
    return copy;
}

public boolean isGoal() {
    return Arrays.deepEquals(board, GOAL);
}

public List<String> getPossibleActions() {
    List<String> actions = new ArrayList<>();
    if (blankRow > 0) actions.add("Up");
    if (blankRow < SIZE - 1) actions.add("Down");
    if (blankCol > 0) actions.add("Left");
    if (blankCol < SIZE - 1) actions.add("Right");
    return actions;
}

public PuzzleState applyAction(String action) {
    int newRow = blankRow;
    int newCol = blankCol;

    switch (action) {
        case "Up": newRow = blankRow - 1; break;
        case "Down": newRow = blankRow + 1; break;
        case "Left": newCol = blankCol - 1; break;
        case "Right": newCol = blankCol + 1; break;
        default: throw new IllegalArgumentException("Invalid action: " + action);
    }

    int[][] newBoard = copyBoard(this.board);
    newBoard[blankRow][blankCol] = newBoard[newRow][newCol];
    newBoard[newRow][newCol] = 0;

    return new PuzzleState(newBoard, action, this, newRow, newCol);
}

public int getManhattanDistance() {
    int distance = 0;
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            int value = board[i][j];
            if (value != 0) {
                int targetRow = (value - 1) / SIZE;
                int targetCol = (value - 1) % SIZE;
                distance += Math.abs(i - targetRow) + Math.abs(j - targetCol);
            }
        }
    }
    return distance;
}

public int getTotalCost() {
    return cost + getManhattanDistance();
}

public int[][] getBoard() { return copyBoard(board); }

```

```

    public String getAction() { return action; }
    public PuzzleState getParent() { return parent; }
    public int getCost() { return cost; }
    public int getBlankRow() { return blankRow; }
    public int getBlankCol() { return blankCol; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        PuzzleState that = (PuzzleState) obj;
        return Arrays.deepEquals(board, that.board);
    }

    @Override
    public int hashCode() {
        return Arrays.deepHashCode(board);
    }
}

// Класс для решения пазла
class PuzzleSolver {
    public static List<PuzzleState> solveAStar(PuzzleState start) {
        java.util.PriorityQueue<PuzzleState> openSet = new
java.util.PriorityQueue<>(
            java.util.Comparator.comparingInt(PuzzleState::getTotalCost)
        );
        java.util.Map<PuzzleState, Integer> gScore = new java.util.HashMap<>();
        java.util.Map<PuzzleState, PuzzleState> cameFrom = new
java.util.HashMap<>();

        openSet.offer(start);
        gScore.put(start, 0);

        while (!openSet.isEmpty()) {
            PuzzleState current = openSet.poll();

            if (current.isGoal()) {
                return reconstructPath(current);
            }

            for (String action : current.getPossibleActions()) {
                PuzzleState neighbor = current.applyAction(action);
                int tentativeGScore = gScore.get(current) + 1;

                if (tentativeGScore < gScore.getOrDefault(neighbor,
Integer.MAX_VALUE)) {
                    cameFrom.put(neighbor, current);
                    gScore.put(neighbor, tentativeGScore);

                    if (!openSet.contains(neighbor)) {
                        openSet.offer(neighbor);
                    }
                }
            }

            return Collections.emptyList();
        }

        private static List<PuzzleState> reconstructPath(PuzzleState goal) {

```

```

        List<PuzzleState> path = new ArrayList<>();
        PuzzleState current = goal;

        while (current != null) {
            path.add(0, current);
            current = current.getParent();
        }

        return path;
    }

    public static boolean isSolvable(int[][] board) {
        int inversions = 0;
        List<Integer> flattened = new ArrayList<>();

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (board[i][j] != 0) {
                    flattened.add(board[i][j]);
                }
            }
        }

        for (int i = 0; i < flattened.size(); i++) {
            for (int j = i + 1; j < flattened.size(); j++) {
                if (flattened.get(i) > flattened.get(j)) {
                    inversions++;
                }
            }
        }

        return inversions % 2 == 0;
    }

    public static int[][] generateRandomState() {
        List<Integer> numbers = new ArrayList<>(Arrays.asList(0, 1, 2, 3, 4, 5,
6, 7, 8));

        Collections.shuffle(numbers);

        int[][] board = new int[3][3];
        int index = 0;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                board[i][j] = numbers.get(index++);
            }
        }

        if (!isSolvable(board)) {
            return generateRandomState();
        }

        return board;
    }
}

// Панель для отображения пазла
class PuzzlePanel extends JPanel {
    private int[][] board;
    private final int tileSize = 80;
    private final int margin = 2;

```

```

public PuzzlePanel(int[][] board) {
    this.board = copyBoard(board);
    setPreferredSize(new Dimension(3 * tileSize, 3 * tileSize));
    setBackground(Color.DARK_GRAY);
}

public void setBoard(int[][] newBoard) {
    this.board = copyBoard(newBoard);
    repaint();
}

private int[][] copyBoard(int[][] original) {
    int[][] copy = new int[3][3];
    for (int i = 0; i < 3; i++) {
        System.arraycopy(original[i], 0, copy[i], 0, 3);
    }
    return copy;
}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2d = (Graphics2D) g;
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            int x = j * tileSize + margin;
            int y = i * tileSize + margin;
            int width = tileSize - 2 * margin;
            int height = tileSize - 2 * margin;

            if (board[i][j] == 0) {
                // Пустая клетка
                g2d.setColor(Color.LIGHT_GRAY);
                g2d.fillRect(x, y, width, height);
                g2d.setColor(Color.DARK_GRAY);
                g2d.drawRect(x, y, width, height);
            } else {
                // Клетка с числом
                g2d.setColor(new Color(70, 130, 180)); // SteelBlue
                g2d.fillRoundRect(x, y, width, height, 15, 15);
                g2d.setColor(Color.WHITE);
                g2d.setFont(new Font("Arial", Font.BOLD, 24));

                String text = String.valueOf(board[i][j]);
                FontMetrics fm = g2d.getFontMetrics();
                int textX = x + (width - fm.stringWidth(text)) / 2;
                int textY = y + (height - fm.getHeight()) / 2 +
fm.getAscent();

                g2d.drawString(text, textX, textY);

                // Обводка
                g2d.setColor(new Color(30, 100, 150));
                g2d.setStroke(new BasicStroke(2));
                g2d.drawRoundRect(x, y, width, height, 15, 15);
            }
        }
    }
}

```

```

    }
}

// Главное окно приложения
public class EightPuzzleSwing extends JFrame {
    private PuzzlePanel puzzlePanel;
    private JButton solveButton;
    private JButton shuffleButton;
    private JButton resetButton;
    private JButton stepButton;
    private JLabel infoLabel;
    private JTextArea solutionArea;

    private int[][] currentBoard;
    private List<PuzzleState> solutionPath;
    private int currentStep;

    public EightPuzzleSwing() {
        initializeUI();
        setupEventHandlers();
        shufflePuzzle();
    }

    private void initializeUI() {
        setTitle("8-Puzzle Game");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout(10, 10));

        // Панель пазла
        currentBoard = new int[3][3];
        puzzlePanel = new PuzzlePanel(currentBoard);
        add(puzzlePanel, BorderLayout.CENTER);

        // Панель управления
        JPanel controlPanel = new JPanel(new FlowLayout());
        solveButton = new JButton("Решить");
        shuffleButton = new JButton("Перемешать");
        resetButton = new JButton("Сброс");
        stepButton = new JButton("Следующий шаг");
        stepButton.setEnabled(false);

        controlPanel.add(solveButton);
        controlPanel.add(shuffleButton);
        controlPanel.add(resetButton);
        controlPanel.add(stepButton);

        // Панель информации
        infoLabel = new JLabel("Нажмите 'Перемешать' чтобы начать");
        infoLabel.setHorizontalAlignment(SwingConstants.CENTER);

        // Область решения
        solutionArea = new JTextArea(8, 20);
        solutionArea.setEditable(false);
        JScrollPane scrollPane = new JScrollPane(solutionArea);
        scrollPane.setBorder(BorderFactory.createTitledBorder("Решение"));

        // Основная панель
        JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
        mainPanel.add(infoLabel, BorderLayout.NORTH);
        mainPanel.add(controlPanel, BorderLayout.CENTER);
        mainPanel.add(scrollPane, BorderLayout.SOUTH);
    }
}

```

```

        add(mainPanel, BorderLayout.SOUTH);

        pack();
        setLocationRelativeTo(null);
        setResizable(false);
    }

    private void setupEventHandlers() {
        shuffleButton.addActionListener(e -> shufflePuzzle());

        solveButton.addActionListener(e -> solvePuzzle());

        resetButton.addActionListener(e -> {
            solutionPath = null;
            currentStep = 0;
            stepButton.setEnabled(false);
            solutionArea.setText("");
            infoLabel.setText("Пазл сброшен");
        });

        stepButton.addActionListener(e -> showNextStep());
    }

    private void shufflePuzzle() {
        currentBoard = PuzzleSolver.generateRandomState();
        puzzlePanel.setBoard(currentBoard);
        solutionPath = null;
        currentStep = 0;
        stepButton.setEnabled(false);
        solutionArea.setText("");
        infoLabel.setText("Пазл перемешан. Нажмите 'Решить' для поиска
решения.");
    }

    private void solvePuzzle() {
        PuzzleState start = new PuzzleState(currentBoard);

        infoLabel.setText("Идет поиск решения...");
        solveButton.setEnabled(false);
        shuffleButton.setEnabled(false);

        // Запускаем в отдельном потоке чтобы не блокировать UI
        SwingWorker<List<PuzzleState>, Void> worker = new SwingWorker<>() {
            @Override
            protected List<PuzzleState> doInBackground() {
                return PuzzleSolver.solveAStar(start);
            }

            @Override
            protected void done() {
                try {
                    solutionPath = get();
                    if (solutionPath.isEmpty()) {
                        infoLabel.setText("Решение не найдено!");
                    } else {
                        currentStep = 0;
                        stepButton.setEnabled(true);
                        infoLabel.setText(String.format("Решение найдено! Шагов:
%d. Нажмите 'Следующий шаг'", solutionPath.size() - 1));
                        displaySolutionSteps();
                    }
                } catch (InterruptedException | ExecutionException e) {
                    e.printStackTrace();
                }
            }
        };
    }

```

```

        }
    } catch (Exception ex) {
        infoLabel.setText("Ошибка при поиске решения");
        ex.printStackTrace();
    } finally {
        solveButton.setEnabled(true);
        shuffleButton.setEnabled(true);
    }
}

};

worker.execute();
}

private void showNextStep() {
    if (solutionPath != null && currentStep < solutionPath.size()) {
        PuzzleState state = solutionPath.get(currentStep);
        puzzlePanel.setBoard(state.getBoard());

        if (currentStep > 0) {
            infoLabel.setText(String.format("Шаг %d/%d: %s", currentStep,
solutionPath.size() - 1,
                        solutionPath.get(currentStep).getAction()));
        }

        currentStep++;

        if (currentStep >= solutionPath.size()) {
            stepButton.setEnabled(false);
            infoLabel.setText("Решение завершено!");
        }
    }
}

private void displaySolutionSteps() {
    StringBuilder sb = new StringBuilder();
    sb.append("Путь решения:\n");

    for (int i = 1; i < solutionPath.size(); i++) {
        PuzzleState state = solutionPath.get(i);
        sb.append(String.format("Шаг %d: %s\n", i, state.getAction()));
    }

    solutionArea.setText(sb.toString());
}

public static void main(String[] args) {
    try {
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
    } catch (Exception e) {
        e.printStackTrace();
    }

    SwingUtilities.invokeLater(() -> {
        new EightPuzzleSwing().setVisible(true);
    });
}
}

```